Type annotations for NumPy arrays

```
TNDArrayInt8 = np.ndarray[Any, np.dtype[np.int8]]
TNDArrayBool = np.ndarray[Any, np.dtype[np.bool_]]
TNDArrayFloat64 = np.ndarray[Any, np.dtype[np.float64]]
def process1(
        v: TNDArrayInt8,
        q: TNDArrayBool,
        ) -> TNDArrayFloat64:
    s: TNDArrayFloat64 = np.where(q, 0.5, 0.25)
    return v * s
```

Using mypy for static analysis of NumPy arrays

```
v1: TNDArrayInt8 = np.arange(20, dtype=np.int8)
x = process1(v1, v1)
# tp.py: error: Argument 2 to "process1" has incompatible type
# "ndarray[Any, dtype[floating[_64Bit]]]"; expected "ndarray[Any, dtype[bool_]]"  [arg-type]
```

Using CallGuard and more-general annotations

```
TNDArrayIntAny = np.ndarray[Any, np.dtype[np.signedinteger[Any]]]
@sf.CallGuard.check
def process3(v: TNDArrayIntAny, q: TNDArrayBool) -> TNDArrayFloat64:
    s: TNDArrayFloat64 = np.where(q, 0.5, 0.25)
    return v * s
```

Using CallGuard for run-time validation of NumPy arrays

```
q: TNDArrayBool = np.arange(20) % 3 == 0
x = process3(v1, q) # no error, same as mypy

v2: TNDArrayInt64 = np.arange(20, dtype=np.int64)
x = process3(v2, q) # no error, same as mypy

v3: TNDArrayFloat64 = np.arange(20, dtype=np.float64) * 0.5
x = process3(v3, q) # error
# static_frame.core.type_clinic.ClinicError:
# In args of (v: ndarray[Any, dtype[signedinteger[Any]]],
# q: ndarray[Any, dtype[bool_]]) -> ndarray[Any, dtype[float64]]
# └── ndarray[Any, dtype[signedinteger[Any]]]
#     └── dtype[signedinteger[Any]]
#         └── Expected signedinteger, provided float64 invalid
```

Type annotations for StaticFrame DataFrames

```
TFrameDateInts = sf.Frame[sf.IndexDate, sf.Index[np.str_], np.int64, np.int64]
TSeriesYMBool = sf.Series[sf.IndexYearMonth, np.bool_]
TSeriesDFloat = sf.Series[sf.IndexDate, np.float64]

def process5(v: TFrameDateInts, q: TSeriesYMBool) -> TSeriesDFloat:
    t = v.index.iter_label().apply(lambda l: q[l.astype('datetime64[M]')]) # type: ignore
    s = np.where(t, 0.5, 0.25)
    return cast(TSeriesDFloat, (v.via_T * s).mean(axis=1))
```

Using mypy for static analysis of StaticFrame DataFrames

```
TFrameDateIntFloat = sf.Frame[sf.IndexDate, sf.Index[np.str_], np.int64, np.float64]
v5: TFrameDateIntFloat = sf.Frame.from_fields([range(5), np.arange(3, 8) * 0.5], columns=('a', 'b'),
index=sf.IndexDate.from_date_range('2021-12-30', '2022-01-03'))

q: TSeriesYMBool = sf.Series([True, False],
index=sf.IndexYearMonth.from_date_range('2021-12', '2022-01'))

x = process5(v5, q)
# tp.py: error: Argument 1 to "process5" has incompatible type
# "Frame[IndexDate, Index[str_], signedinteger[_64Bit], floating[_64Bit]]"; expected
# "Frame[IndexDate, Index[str_], signedinteger[_64Bit], signedinteger[_64Bit]]"  [arg-type]
```

Using CallGuard for run-time validation of StaticFrame DataFrames

```
# a Frame of three columns of integers
TFrameDateIntIntInt = sf.Frame[sf.IndexDate, sf.Index[np.str_], np.int64, np.int64, np.int64]
v6: TFrameDateIntIntInt = sf.Frame.from_fields([range(5), range(3, 8), range(1, 6)],
columns=('a', 'b', 'c'), index=sf.IndexDate.from_date_range('2021-12-30', '2022-01-03'))

x = process5(v6, q)
# static_frame.core.type_clinic.ClinicError:
# In args of (v: Frame[IndexDate, Index[str_], signedinteger[_64Bit], signedinteger[_64Bit]],
# q: Series[IndexYearMonth, bool_]) -> Series[IndexDate, float64]
# └── Frame[IndexDate, Index[str_], signedinteger[_64Bit], signedinteger[_64Bit]]
#     └── Expected Frame has 2 dtype, provided Frame has 3 dtype--
```

Modern Python type annotations can be used with arrays and DataFrames to improve code quality through static analysis and runtime validation

Improving Code Quality with Array and DataFrame Type Hints

Christopher Ariza
Research Affiliates

As tools for Python type annotations (or hints) have evolved, more complex data structures can be typed, improving maintainability and static analysis. Arrays and DataFrames, as complex containers, have only recently supported complete type annotations in Python. NumPy 1.22 introduced generic specification of arrays and dtypes. Building on NumPy's foundation, StaticFrame 2.0 introduced complete type specification of DataFrames, employing NumPy primitives and variadic generics. This article demonstrates practical approaches to fully type-hinting arrays and DataFrames, and shows how the same annotations can improve code quality with both static analysis and runtime validation.

Associating Type Hints with the Structure of a DataFrame and a Series

The following diagram uses color to associate type annotations in a function signature to a diagram of a DataFrame (including index, column, and value types) and a Series (including hierarchical index types and value types).